Python Grundlagen Kontrollstrukturen

Entscheidungen und Wiederholungen

Studienkolleg · Rahn Education

Was sind Kontrollstrukturen?

Wichtige Wörter

Begriffe für diese Lektion

- **Kontrollstruktur:** Steuert den Ablauf eines Programms
- **Bedingung:** Ein Ausdruck, der `True` oder `False` ergibt
- **Schleife:** Wiederholt Code mehrmals
- **Einrückung:** In Python Pflicht – zeigt zusammengehörigen Code

Definition

Kontrollstrukturen

Bedingte Anweisungen: Entscheidungen treffen (`if`/`else`)

Schleifen: Wiederholungen (`for`, `while`)

Sprunganweisungen: Ablauf unterbrechen (`break`, `continue`)

Die if-Anweisung

if – Entscheidungen treffen

Syntax

Führt Code aus, wenn eine Bedingung erfüllt ist.

```
if Bedingung:
    Anweisung
```

Beispiel

Einfache if-Anweisung

```
1 alter = 19
2
3 if alter >= 18:
4     print("Du bist volljaehrig.")
```

Beispiel

Einfache if-Anweisung

```
alter = 19

if alter >= 18:
    print("Du bist volljaehrig.")
```

```
Du bist volljaehrig.
```

Beispiel

if-else: Zwei Möglichkeiten

```
1 alter = 16
2
3 if alter >= 18:
4     print("Du bist volljaehrig.")
5 else:
6     print("Du bist noch nicht volljaehrig.")
```

Beispiel

if-elif-else: Mehrere Bedingungen

```
note = 2.8

if note <= 1.5:
    print("Sehr gut!")
elif note <= 2.5:
    print("Gut!")
elif note <= 3.5:
    print("Befriedigend!")
elif note <= 4.0:
    print("Ausreichend!")
else:
    print("Nicht bestanden!")
```

Übung

Vorzeichen prüfen

Schreibe ein Programm mit `if-elif-else`:

1. Zahl größer als 0 → `"Positiv"` ausgeben
2. Zahl kleiner als 0 → `"Negativ"` ausgeben
3. Sonst → `"Null"` ausgeben

Tipp: `zahl = int(input('Zahl eingeben: '))`

Operatoren

Wichtige Wörter

- **Vergleichsoperator:** Vergleicht zwei Werte (`==`, `!=`, `>` ...)
- **Logischer Operator:** Verknüpft Bedingungen (`and`, `or`, `not`)
- **Boolean:** Wahrheitswert – `True` oder `False`

Vergleichsoperatoren

Übersicht

`==`	gleich
`!=`	ungleich
`>`	größer als
`<`	kleiner als
`>=`	größer oder gleich
`<=`	kleiner oder gleich

Beispiel

Logische Operatoren

```
alter = 25
einkommen = 2000

if alter >= 18 and einkommen > 1500:
    print("Beide Bedingungen erfuellt.")
```

Beispiel

Logische Operatoren

```
alter = 25
einkommen = 2000

if alter >= 18 and einkommen > 1500:
    print("Beide Bedingungen erfuellt.")
```

```
if alter > 16 or einkommen < 1000:
    print("Mindestens eine Bedingung erfuellt.")
```

Beispiel

Logische Operatoren

```
alter = 25
einkommen = 2000

if alter >= 18 and einkommen > 1500:
    print("Beide Bedingungen erfuellt.")
```

```
if alter > 16 or einkommen < 1000:
    print("Mindestens eine Bedingung erfuellt.")
```

```
if not alter < 18:
    print("Du bist nicht minderjaehrig.")
```

Die for-Schleife

for – Sequenzen durchlaufen

Syntax

Wiederholt Code für jedes Element einer Sequenz.

```
for Element in Sequenz:
    Anweisung
```

Beispiel

Liste durchlaufen

```
1 fruechte = ["Apfel", "Banane", "Kirsche"]
2
3 for frucht in fruechte:
4     print(frucht)
```

Beispiel

Liste durchlaufen

```
1 fruechte = ["Apfel", "Banane", "Kirsche"]
2
3 for frucht in fruechte:
4     print(frucht)
```

```
Apfel
Banane
Kirsche
```

Beispiel

Die range()-Funktion

```
1 # range(stop): 0 bis 4
2 for i in range(5):
3     print(i)          # 0, 1, 2, 3, 4
```

Beispiel

Die range()-Funktion

```
# range(stop): 0 bis 4
for i in range(5):
    print(i)          # 0, 1, 2, 3, 4
```

```
# range(start, stop): 1 bis 6
for i in range(1, 7):
    print(i)          # 1, 2, 3, 4, 5, 6
```

Die while-Schleife

while – Bedingte Wiederholung

Syntax

Wiederholt Code, solange eine Bedingung erfüllt ist.

```
while Bedingung:
    Anweisung
```

Beispiel

Zähler mit while

```
1 zaehler = 0
2
3 while zaehler < 5:
4     print(zaehler)
5     zaehler += 1     # Wichtig: Zaehler erhoehen!
```

Beispiel

Zähler mit while

```
1 zaehler = 0
2
3 while zaehler < 5:
4     print(zaehler)
5     zaehler += 1     # Wichtig: Zaehler erhoehen!
```

```
0  1  2  3  4
```

Häufiger Fehler

Endlosschleife

```
zaehler = 0

while zaehler < 5:
    print(zaehler)
    # zaehler += 1 vergessen -> Endlosschleife!
```

Häufiger Fehler

Endlosschleife

```
zaehler = 0

while zaehler < 5:
    print(zaehler)
    # zaehler += 1 vergessen -> Endlosschleife!
```

Vergiss `zaehler += 1` nicht!

Sonst läuft das Programm **endlos**.

Übung

Schleifen anwenden

1. Gib alle geraden Zahlen von 2 bis 20 mit `for` aus.
2. Schreibe eine `while`-Schleife, die von 10 auf 0 herunterzählt.
3. Gib alle Elemente der Liste `["Mo", "Di", "Mi", "Do", "Fr"]` aus.

break und continue

Sprunganweisungen

Übersicht

`break` $\rightarrow$ Schleife sofort beenden

`continue` $\rightarrow$ Aktuellen Durchlauf überspringen

Beispiel

break – Schleife beenden

```
1 for i in range(1, 11):
2     if i == 5:
3         break        # Beendet bei 5
4     print(i)
```

Beispiel

break – Schleife beenden

```
1 for i in range(1, 11):
2     if i == 5:
3         break        # Beendet bei 5
4     print(i)
```

```
1
2
3
4
```

Beispiel

continue – Durchlauf überspringen

```
1 for i in range(1, 6):
2     if i == 3:
3         continue     # Ueberspringt 3
4     print(i)
```

Beispiel

continue – Durchlauf überspringen

```
1 for i in range(1, 6):
2     if i == 3:
3         continue     # Ueberspringt 3
4     print(i)
```

```
1
2
4
5
```

Verschachtelte Schleifen

Beispiel

Multiplikationstabelle

```
1 for i in range(1, 4):
2     for j in range(1, 4):
3         print(i, "x", j, "=", i*j)
4     print()      # Leere Zeile nach jeder Reihe
```

Häufige Fehler

Häufige Fehler

SyntaxError und IndentationError

```
# Fehler 1: = statt ==
if x = 5:              # SyntaxError!
    print("x ist 5")
```

Häufige Fehler

SyntaxError und IndentationError

```
# Fehler 1: = statt ==
if x = 5:              # SyntaxError!
    print("x ist 5")
```

```
# Fehler 2: Einrueckung vergessen
if x == 5:
print("x ist 5")       # IndentationError!
```

Häufige Fehler

SyntaxError und IndentationError

```
# Fehler 1: = statt ==
if x = 5:              # SyntaxError!
    print("x ist 5")
```

```
# Fehler 2: Einrueckung vergessen
if x == 5:
print("x ist 5")       # IndentationError!
```

```
# Fehler 3: Doppelpunkt vergessen
if x == 5              # SyntaxError!
    print("x ist 5")
```

Übung

Fehlersuche – Finde und korrigiere die Fehler

```
zahl = 10
if zahl % 2 = 0:
    print("Gerade")

summe = 0
for i in range(1, 6)
    summe += i
print(summe)
```

Große Übung

Übung

Klausurvorbereitung – Was gibt dieser Code aus?

```
1 for i in range(5):
2     if i == 2:
3         continue
4     if i == 4:
5         break
6     print(i)
```

Tipp: `continue` überspringt den Rest, `break` beendet die Schleife.

Bedingte Anweisungen

a) Schreibe ein Programm, das eine Note einliest und die Bewertung ausgibt (sehr gut / gut / befriedigend / ausreichend / nicht bestanden). *(5P)*

b) Welcher Fehler steckt im folgenden Code? *(2P)*

```
if x = 10:
print('Zehn')
```

c) Was gibt `not (5 > 3)` aus? *(1P)*

Schleifen

a) Was gibt folgender Code aus? *(3P)*

```
for i in range(5):
if i == 2: continue
if i == 4: break
print(i)
```

b) Schreibe eine `while`-Schleife, die alle Vielfachen von 3 von 3 bis 30 ausgibt. *(4P)*

c) Was ist der Unterschied zwischen `break` und `continue`? *(3P)*

Fehleranalyse und Korrektur

a) Finden und korrigieren Sie alle Fehler: *(6P)*

```
zaehler = 0
while zaehler < 3
print(zaehler)
```

b) Schreibe ein Programm mit einer `for`-Schleife, das die Summe aller Zahlen von 1 bis 100 berechnet und ausgibt. *(5P)*

c) Erkläre, warum Einrückung in Python wichtig ist. *(2P)*

Fragen?